SUPER: Evaluating Agents on Setting Up and Executing Tasks from Research Repositories

Anonymous ACL submission

Abstract

Given that Large Language Models (LLMs) have made significant progress in writing code, can they now be used to autonomously reproduce results from research repositories? Such a capability would be a boon to the research community, helping researchers validate, understand, and extend prior work. To measure this, we present SUPER, the first benchmark on setting up and executing tasks from repositories. To capture realistic tasks facing researchers, we derive tasks from 42 research repositories in Machine Learning (ML) and Natural Language Processing (NLP) research papers. Each task is annotated with an expert solution which is used to create a benchmark of 152 *scenarios*. Each scenario captures specific challenges associated with reproduction of experiments (e.g. configuring a trainer). We develop two evaluation measures to measure task success as well as progress towards the task. We show that state-of-the-art approaches struggle to solve these scenarios with the best model (GPT-4o) solving only 38.3% of the scenarios. This illustrates the challenges still present in this important task, and suggests that SUPER can serve as a valuable resource for the community to make and measure progress.[1]

1 Introduction

Research and scientific discoveries often rely on the reproducibility of experiments conducted by other researchers and the ease with which scientists can build upon each other's work. In the context of empirical ML and NLP research, it is often crucial for researchers to be able to execute and reproduce diverse research experiments from open-source repositories, whether to verify existing results or to test them under new conditions.

In practice, even when research code is available, running arbitrary code repositories is often non-trivial and time-consuming. Executing experiments frequently requires substantial effort to set up the environment, including non-trivial configuration changes, resolving outdated package dependencies, fixing bugs, and determining the correct execution commands, among other tasks. All of these tasks require a considerable understanding of the documentation and repository code, knowledge about fixing standard issues (e.g., CUDA issues) as well as the ability to modify the code appropriately. These steps are especially time-consuming for research repositories "in-the-wild", as support and documentation may not be available.

In this work, we ask: *Can LLMs automate the set up and execution of tasks in research repositories?* Consider the research task in Fig. 1 where the agent has to use a repository from a paper to train a model with a new optimizer, and evaluate its performance on a custom dataset. To success-



Figure 1: An illustration of a research task and some of the steps an agent would need to complete it, including updating data loading configuration, resolving dependency issues (due to unlisted version dependencies in the repository), running the training script and reporting metrics. Our scenarios are derived from such end-to-end research tasks by masking certain steps.

[1] Code and data to be released under CC by 4.0 license.

Resource	SUPER (this work)	DS-1000 (Lai et al., 2023)	ML-Bench (Liu et al., 2023b)	MLAgentBench (Huang et al., 2024)	SWE-bench (Jimenez et al., 2024)
Repo. understanding	✓	✗	✓	✗	✓
Multi-File edits	✓	✗	✗	✗	✓
Partially-observed environment	✓	✗	✗	✓	✗
Requires repository setup	✓	✗	✗	✗	✗
# source repositories (Median stars)	**42** (11)	8 (34,589)	18 (8,991)	NA	12 (12,309)
# problems	154 (+42)	1000	9641	15	2300

Table 1: Comparison of SUPER against four other related code execution benchmarks in terms of the challenges being tested (rows 1-4) and the number of source repositories and problems in the dataset (row 5-6).

fully complete the task, the agent would need to install dependencies, download the provided data and make code changes to load the data, and then execute the training script (first four gray cells in the figure). It has to respond to unexpected issues, such as the incompatible dependency (fourth cell), resolve it by downgrading a dependency (fifth cell) and finally report the result metrics (last cell).

While LLM-based agents have recently been used to produce execution commands from popular research repositories (Liu et al., 2023b), conduct ML experiments in simple environments (Huang et al., 2024) or resolve repository issues (Jimenez et al., 2024), no existing benchmark evaluates agents on the common problem faced by many researchers: both *setting up and executing* experiments using research repositories in-the-wild, where code is often not well-maintained and experiment settings are not easily configurable.

To encourage research on this problem, we introduce **SUPER** (**S**etting **UP** and **E**xecuting tasks from **R**esearch repositories), a benchmark comprising 152 *scenarios* commonly encountered when running experimental tasks in *low-profile* research repositories. Each scenario addresses a specific challenge, such as installing dependencies and resolving conflicts, configuring experimental data, setting up hyper-parameters, resolving runtime exceptions, correctly executing scripts, and others. We derive these scenarios by first collecting 42 expert solutions for research-related *end-to-end tasks* based on a diverse set of repositories. We extract scenarios using our proposed "Code Masking" mechanism, loosely inspired by cloze tests (Taylor, 1953) and masked language models (MLM) (Devlin et al., 2019), where we remove parts of the expert-written code to generate a diverse set of scenarios targeting a specific challenge.

To solve these scenarios, agents are given a specific goal to complete and a final metric to provide. Since there is often more than one way to achieve these goals, evaluating agents based solely on their actions is unreliable. Instead, we use both an *outcome-based evaluation*, where agents are evaluated on getting correct outputs, and a finer-grained *landmark-based evaluation*, which checks for specific 'landmark' states in the agents' solutions (e.g., reaching a 'training completed' stage) to allow for partial credit to be given for making reasonable progress toward solving the task.

We evaluate both proprietary and open-source models on SUPER as the underlying models of strong baseline agents with access to file-editing tools. We find that agents struggle to correctly solve many of the scenarios, with only 38.3% of the scenarios being solved correctly by our strongest agent. These agents are even further away from solving entire research tasks, completing correctly only 7.1% of the *end-to-end* tasks. Open-source models entirely fail on both the scenarios and end-to-end tasks. Through analysis of model trajectories, we find that agents are better at resolving well-specified scenarios, such as solving exceptions, bugs, and other issues, than at scenarios that require going through the repository and files to understand the structure of code. These results underscore many of the core challenges facing LLM-based experiment execution systems, which our benchmark aims to help advance.

2 Related Work

Coding benchmarks: While early code benchmarks (Chen et al., 2021; Austin et al., 2021; Cassano et al., 2022) mainly focused on synthesizing simple functions from descriptions, recent benchmarks have shifted to more complex competitive programming problems (Li et al., 2022; Hendrycks et al., 2021; Jain et al., 2024) and evaluating proficiency with popular data science libraries (Lai et al., 2023). Unlike these, we follow the recent trend on evaluating LLMs in more natural programming scenarios, such as programming with external tools and APIs (Li et al., 2023; Shen et al., 2023; Wang et al., 2023b), code editing and debugging (Cassano



Figure 2: An overview of the end-to-end tasks (middle) and scenarios construction (right) pipeline of SUPER.

et al., 2023; Tian et al., 2024; Haque et al., 2023), resolving GitHub issues (Jimenez et al., 2024), understanding and coding within a repository context (Liu et al., 2023a; Ding et al., 2024; Zhang et al., 2023), and automating simple ML tasks (Liu et al., 2023b; Huang et al., 2024).

In contrast to these works, SUPER focuses on the end-to-end task of setting up and executing research tasks in repositories, presenting a unique set of challenges. Table 1 compares the four datasets most relevant to SUPER. Tasks require repository comprehension and reasoning, editing multiple files, setting up the repository environment for execution, running commands, and making sequential decisions while interacting with the environment. MLAgentBench (Huang et al., 2024), the closest prior work, evaluates LLMs' ability to run simple ML experiments but focuses on optimizing ML experiments rather than comprehending and setting up a repository for experimentation.

LLM Agents: Recent advancements in LLM-based agents have shown significant progress across various domains, including video games (Wang et al., 2023a), web navigation (Yao et al., 2022; Zhou et al., 2023), human interaction simulation (Park et al., 2023), automating complex computer tasks (Xie et al., 2024), data science and machine learning (Guo et al., 2024; Hong et al., 2024; Liu et al., 2024; Yang et al., 2024b), open-ended discovery (Jansen et al., 2024), and coding (Wang et al., 2024; Yang et al., 2024a; OpenDevin Team, 2024). Our benchmark introduces an important new domain that encourages the development of LLM-based agents to assist researchers in their tasks.

3 Benchmark Construction

In this section we describe the process of building the SUPER benchmark. As a high-level overview (Fig. 2), we collect tasks following these steps: (1) We identify a set of relevant code repositories from research papers and write research-oriented tasks based on them (§3.1). (2) We collect end-to-end solutions for these tasks from human experts (§3.2). (3) We extract multiple scenarios for each task based on the experts' solutions (Masked Code Scenarios, §3.3). (4) We create two types of evaluation for each scenario: *outcome-based*, where we compare the agent's answer to the gold answer, and a more lenient *landmark-based* evaluation that indicates progress toward correctly solving the task, even if the solution is not entirely correct (§3.4).

Environment Setup. Running research-oriented repositories often necessitates both being able to run system shell commands (e.g. to install dependencies and run scripts) and stateful Python commands. Previous work and environments typically support only one of these (e.g. only system shell commands (Jimenez et al., 2024) or only Python commands (Huang et al., 2024)). Instead, we build an environment that allows running both of these commands with a Jupyter notebook as engine. In this setup, each execution code is equivalent to running a notebook *cell*, which contains Python code and/or bash commands, and where state is reserved between cell executions (e.g. each cell can use any of the previously defined Python variables). The execution of each cell returns an observation string.

3.1 Tasks

We collect tasks with the following two common settings as leading motivation: (1) reproducing numbers from research papers by running specific experiments, and (2) running modified experiments with different datasets, models, or configs.

Repository Selection. We start by collecting repositories from the "Papers With Code" (`github.com/paperswithcode/paperswithcode-data`) database, which contains research papers linked to their GitHub repositories, along with some additional metadata such as the modality of the datasets used. We only sample research papers with "Text" modalities and select repositories from 2021 or beyond.

Task Generation. We then manually review the sampled repositories and write tasks that involve running a single experiment that is mentioned either in the repository's "readme" file or under a script available in the repository, if such can be found. Whenever possible, we make the task more challenging by requiring the experiment to be run on a new dataset or model, other than the one described in the available documentation. In these cases, we select either datasets available on

HuggingFace Hub (https://huggingface.co/datasets) or provide a google drive link where the dataset can be found. The challenge of running on a specific dataset varies in difficulty: it could involve only a single configuration line change if the dataset is already supported, or creating a new dataset reader, adjusting column names, etc.

For each task, we define (1) the target Github repository, (2) the task definition (e.g. "train a model..."), (3) the metrics or output to be reported (e.g. "F1 metric on the validation set"), along with a specific structure of how the answer should be formatted, and (4) implementation instructions (e.g. specific hyper-parameters). The implementation instructions are important for two reasons: first, to allow fair evaluation of submitted answers by making sure the task is not under-specified, such that two agents that correctly complete the task get the same results. Second, to minimize computational requirements, as described next.

Minimizing Computational Requirements. To make SUPER faster and cheaper to run, we ensure tasks are executable without reliance on GPU machines and that they do not require more than 10 minutes of compute (e.g., for training models or installing packages) on basic compute instances (see §4 for compute details). We therefore create tasks that only require minimal compute by only asking to train and evaluate small models (e.g., 'gpt2-small'), and by adding implementation instructions to the task, such as "only load the first 10 examples of the dataset" or "run a single epoch".

Note that these restrictions do not make the task any easier for the agent. In fact, they often add additional challenges that agents need to solve (e.g., configuring hyper-parameters, finding where data is loaded to limit its loading to the first 10 samples, being able to run experiments that were designed for GPU on a CPU, etc.).

3.2 Expert Annotation

We use Upwork (https://www.upwork.com/) to find and hire experts that have experience with running ML and NLP experiments. We filter the initial list of applications by testing them on a pilot task which we solved ourselves, to make sure they are able to correctly execute a task and effectively solve issues by comparing their solution and results to ours. We instruct workers to execute their solutions on Google Colab, allowing us to collect the solutions in a consistent notebook-like environment.

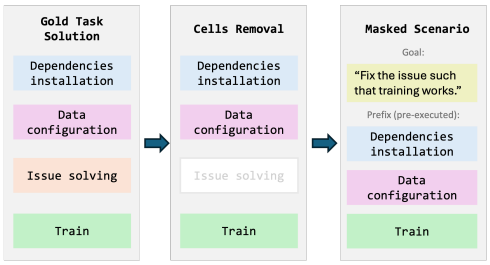


Figure 3: An abstract demonstration of how scenarios are extracted: starting from a gold end-to-end task solution (left), we remove cells (middle) that focus on certain aspects (differently colored cells in the figure), then create a masked problem by defining a goal and prefix cells (right). The prefix cells are executed in the environment, and the agent must then write code to solve the scenario.

We ask the experts to submit their (1) solution notebook, (2) answers (i.e. metrics to be reported), the specific git commit hash of the repository that they have used, and the final version list of all dependencies that were installed throughout the notebook.[2] In addition, we instruct them to use default parameters whenever possible, and to report any decision that they had to make but was not specified in the original task. We add these decisions to the task description or implementation instructions to ensure that any agent solving the same task would have all the necessary information needed to get the same results. For example, the selection of a delimiter token or specific hyper-parameters when no default values are provided in the repository.

Finally, we manually review their solutions, making sure that (1) the solution correctly follows the task, (2) it can be executed in our environment, (3) all unspecified decisions have been recorded and (4) re-running the experiment multiple times yields the same results (up to an error of 10^{-2}). If needed, we ask the workers to make corrections, or manually fix issues ourselves. We provide cost details and guidelines in Appendix B.

3.3 Masked Coding Scenarios Extraction

Solving an end-to-end execution task can often be long and complex, consisting of multiple non-trivial steps. As such, evaluating agents on their ability to run an entire task provides a sparse signal of success, where agents have to complete numerous steps correctly to succeed, making it harder to "hill-climb" results. Instead, we want to evaluate models in a more fine-grained way that will allow us to get success signals for any incremental

[2]The git hash and dependencies ensure that these solutions can be reproduced in the future even as repository and package versions change.

progress towards the task. To this end, we propose to focus on a specific problem (scenario) from the task solution at a time.

We turn to an approach loosely inspired by cloze tests (Taylor, 1953) and masked language models (MLM; Devlin et al., 2019): given a gold solution for a task we remove (*mask*) some part of it (e.g., code that solves a dependency installation issue), and define the scenario such that the agents only needs to solve this narrower aspect of the overall task, as captured by the removed cells.

While the ideal goal is for the agents to complete tasks in an end-to-end manner, extracting masked scenarios allow us to evaluate agents on a broad range of technical challenges often encountered when working with research repositories, while performing a finer-grained analysis of their performance. In addition, this setup aligns well with the usage of interactive code assistants (e.g., CoPilot and Colab's AI-powered coding), where agents assist users that have already written or even executed some code, and can specify what issue or problem remains to be solved.

Masked Coding Scenarios. Each masked scenario consists of (1) a textual description of the remaining tasks to execute (for example "fix the runtime error to complete training" after removing cells from the gold solution that fix an issue), and (2) code prefix (e.g. other cells from the original notebook). Given a masked scenario, the code prefix is pre-executed in the environment, and must then execute their own cells to solve the scenario.

Extraction Procedure. We extract masked coding scenarios in a manual process where we first identify certain cells (not necessarily consecutive) in the gold solution that focus on a certain aspect. For example, the cells corresponding to loading and modifying the dataset could be combined into a *data configuration* block as shown in pink in Fig. 3. We extract the scenarios by first masking a block (*Issue solving* orange block in the figure). We then identify cells that do not depend on the masked block and define a goal that is remaining to be completed (e.g. making training run in our example). These cells are pre-executed and the agent's task is to complete the goal.

We use the masked block and goal cell to define the scenario, e.g. if the code to handle execution on CPU has been masked, the scenario definition would be "*I have already executed some of the required steps. Now, you should make the necessary changes to make sure the code runs on a CPU. Your goal is to successfully run 'train.py'.*". We choose cells with clearly identifiable success indicators as goals, e.g., successfully running 'train.py' would produce metrics on completion. We rely on these outputs for evaluation of correctness (see landmark outputs in §3.4 for details).

3.4 Evaluation

Accuracy Evaluation. As we describe in §3.1, experts provide us a deterministic solution for each task, which we then execute in our environment to get the gold answer, which allow us to evaluate agents based on their outcome. Answers consist of several values (e.g. numbers for metrics, string for model predictions), thus for each scenario, that accuracy metric is the portion of correctly answered values, that is, where the predicted answer precisely matches the gold one (up to a 10^{-2} error). Unlike reference based evaluation used in cloze tests and many prior coding benchmarks (Liu et al., 2023a,b), outcome-based evaluation does not require code to match the gold cells and allows for alternate valid solutions. Also an incorrect solution even with sufficient overlap with the gold solution would still be penalized, for example, skipping just one hyper-parameter will most likely result in the wrong answer.

Landmark-Based Evaluation. Sometimes an indication of whether the model was precisely correct may be too strict, "punishing" models that make progress but don't reach the end. E.g., an agent that loads the data but doesn't train would have the same accuracy as an agent that fails at the start.

To measure progress towards the final goal, we use the gold task notebooks to identify *landmark outputs*; outputs from the environments that act as "evidence" that a particular step was run successfully. E.g., the explicit output string "`*** training completed ***`" in Figure 1 or the string "`Saving dummy inputs... 100%`" implying successful data loading.

Importantly, a perfect landmark score does not entail a perfect accuracy score, as landmarks only indicate that some action was performed, but it was not necessarily correct (e.g. a training script run successfully but with wrong hyper-parameters could be counted as success). Similarly, albeit unlikely by design, a model could correctly solve the task but not hit all of the landmarks (e.g. if it uses an alternate approach or guesses a solution) and

Category (%)	Portion	Gold LOC	Example(s) description of a gold solution
Dependencies	19.7%	4.1	Downgrade ‘transformers‘ version to allow execution of an older repository
CPU	7.2%	5.1	Remove ‘`.cuda()`‘ from different locations in code
Configuration	12.5%	8.2	Edit Python or shell scripts to set hyper-parameters and experiment details
Data	23.7%	22.7	Download custom dataset, update data loader, limit to loading first 10 samples
Issue	9.2%	5.8	Pytorch incompatible tensor shapes; incorrectly loaded Python package
Goal	25.0%	6.5	Run the evaluation script then load generated file to report metrics
Other	2.6%	3.8	Save the model after training to allow evaluation loading

Table 2: Distribution of scenario categories and description of representative solutions from experts. LOC stands for lines of code, counting the number of lines (excluding comments) in the gold solution.

have a lower landmark score. For each gold solution we manually extract 2-6 landmark outputs patterns. The landmarks metric evaluates the percentage of these patterns that appear in the outputs of any of the cells executed by the agent.

3.5 SUPER Benchmark

SUPER consists of 152 masked coding scenarios derived from 42 research code repositories (for a full list of repositories see Table 7). Each scenario is paired with a gold output for outcome-based evaluation and a small set of landmarks for our softer evaluation metric (3 average landmarks per scenario). To verify that the repositories used in our benchmarks are indeed ‘low-profile’, we use the (median) number of GitHub stars in the source repositories as a proxy for popularity as shown in Table 1. Intuitively, popularity loosely correlates with the quality of documentation and readiness. We see that the median number of stars for our repositories (11) is considerably lower than other comparable datasets (see Appendix A for details).

Table 2 shows the distribution of the extracted scenarios across various categories, along with a representative solution for each category and the average lines of code (LOC) that were executed in the gold solution. LOC is counted with the number of lines (excluding comments) in the gold solution, and for editing cells, the number of changed lines. Note that the gold LOC can be dramatically lower than those needed by the agent; experts did not have to write code to read contents of files in the repository as they can use their IDE or browser, while agents have to browse files through an explicit command. In addition, experts did not necessarily keep cells with failed attempts.

End-to-End Tasks. Finally, we also release SUPER-E2E, the end-to-end version of the benchmark based on the 42 collected tasks, where agents are evaluated on successfully solving the entire task, using the same final answers and landmarks used in SUPER. The length of the gold solutions (with an average of 44.3 LOC and 14.4 cells per solution) makes these tasks particularly challenging due to the long agent trajectories required. Consequently, the ability to solve these tasks provides an important signal on the performance of agents in handling long and complex tasks.

4 Experiments

Experimental Setup We limit the execution time (i.e. time to run commands, not counting the API reply time) of each scenario to 30 minutes. We run all tasks on compute instances using sandboxed execution in Modal (`https://modal.com`), which allows us to evaluate scenarios safely and concurrently, speeding up evaluations. We limit the total input tokens number (summing all steps) of each scenario to 400k, and of each end-to-end task to 600k. The compute cost for executing each scenario in Modal (not counting API usage) is 2-3 cents, making it negligible compared to API costs. If time or token limit occurs before the agent submits the answer, the task is concluded without submission (in such cases, agents will only be scored based on the landmark evaluation). We validate that running the gold trajectories yield perfect scores on all of our metrics in three consequent attempts. For the E2E tasks, we average results over 3 seeds.

Underlying LLMs We experiment with a mixture of agents based on the commerical LLMs, GPT-4o (`gpt-4o-2024-05-13`), GPT-4-turbo (`gpt-4-turbo-2024-04-09`) (OpenAI, 2023), and compare against the open-source model Mixtral-8x22B-Instruct (Jiang et al., 2024) that was selected due to its ability to work effectively with our JSON prompt format (see Appendix C).

4.1 Baselines

In this section, we describe the three baseline agents that we evaluate on SUPER: ReAct (Yao et al., 2023), our improved version ReAct-SUPER , and

LLM / Agent	Acc.	Subm.	Landm.
GPT-4o			
SWE-agent	23.7	33.6	42.6
ReAct	32.9	50.0	56.7
ReAct-SUPER	34.9	55.3	64.7
GPT-4-turbo			
SWE-agent	18.4	28.9	34.8
ReAct-SUPER	33.6	51.3	57.0
Mixtral			
SWE-agent	0.0	0.0	0.0
ReAct-SUPER	0.0	0.0	0.0

Table 3: Results of our baselines on SUPER with different underlying LLMs.

Agent	Acc.	Subm.	Landm.
ReAct-SUPER	7.1	26.2	29.3
SWE-agent	4.2	19.0	24.6

Table 4: Results on SUPER-E2E with GPT-4o as the underlying LLM, averaged across three seeds.

SWE-Agent (Yang et al., 2024a). All of our agents are given access to a Jupyter notebook environment where they can **execute** Python or Bash commands (preceded with a '!' character). Other than the **execute** action, they can also **submit** an answer when done. For each scenario we execute the provided 'pre-execute' cells (§3.3) and pass them as an existing history of actions to each agent. With end-to-end tasks, only the task description is passed.

ReAct (Yao et al., 2023) is a baseline agent that iteratively prompts the underlying LLM to output both an action and a natural language "thought" given the interaction history. The action is executed against our environment and a <thought, action, observation> tuple is added to the history.

One challenge associated with running experiments is that output observations can get extremely long (e.g. the output of certain training scripts and dependency installation reaches between 10k-40k tokens). ReAct agents "accumulate" history information (thought, action, and observation triplets) at each step, which makes token usage grow rapidly. As agents are typically limited to a fixed budget (either cost or tokens), this often leads to agents failing to solve the scenario. We apply truncation strategies in all our ReAct agents to mitigate this issue. See Appendix C for details.

ReAct-SUPER The ability to execute Python and bash commands, in theory, allows agents to perform any necessary task. However, these actions are still limited compared to humans who can use IDEs to browse and edit files. In our early experiments, we indeed found that agents struggle to edit files (e.g. change configs) using just bash.

To address this challenge, we supplement the agent with an additional **edit** action, similar in spirit to the *Agent-Computer Interfaces* from SWE-Agent (Yang et al., 2024a). Specifically, the edit command accepts three parameters: the name of the file, the exact content of the lines to be replaced, and the content to replace it with. Notably, we do not need the agent to provide line numbers (as needed by git patch or SWE-Agent) which tends to be error-prone. See App. D for more details.

Finally, as a simple heuristic to show the importance of handling long observations, we filter out any lines that appear to contain progress indications based on pre-defined pattern matching.

SWE-Agent (Yang et al., 2024a) is a ReAct-based agent, originally designed to solve GitHub *issues*. Like ReAct-SUPER , this approach provides agents with tools that allow easier reading of files and scrolling through their contents, highlights syntax errors when editing files and more (see original paper for details). We implement this agent in our environment and modify the prompt to address the execution of research tasks and our specific environment.

4.2 Results

In addition to the accuracy and landmarks metrics (§3.4), we also measure the *submission rate*: the portion of scenarios for which the agent submitted any answer. Agents are instructed to submit an empty answer in case they have finished but have nothing to submit, thus only agents that have submitted a non-empty answer *and* have not reached their token or time limit are considered.

We show results in Table 3 on the scenarios set, demonstrating that ReAct-SUPER correctly solves a significant portion (34.9%) of the challenges that are required to set-up and execute experiments from research repositories, but that most scenarios are still unsolved. The higher landmarks evaluation score (64.7%) suggests that agents often still make progress towards solving the scenarios, even if not necessarily correctly.

SWE-agent performs worse than ReAct-SUPER , and also submits less answers. This happens due to more scenarios reaching token limit (68.4% vs 37%), often due to repeated failed editing loops. Notably, GPT-4-turbo agents shows a consistent drop in metrics compared to GPT-4o. Finally, the

Agent	Acc.	Subm.	Landm.
ReAct-SUPER	34.9	55.3	64.7
Reflexion (Shinn et al., 2023)	38.3	63.2	68.3

Table 5: Results of our best agent (ReAct-SUPER with GPT-4o) with and without Reflexion. While retrying with reflection does help improve the submission rate and accuracy, SUPER benchmark still remains challenging.

Agent	Acc.	Subm.	Landm.
ReAct-SUPER	34.9	55.3	64.7
- edit	34.2	55.0	62.6
- progress lines removal	36.8	56.6	61.9

Table 6: Ablations of two key features of the ReAct-SUPER agent (using GPT-4o). While both edit and progress lines removal are useful on the landmark score, we observe mixed results on the other metrics.

open-source Mixtral completely fails to solve any scenario, and in most cases does not go beyond simple attempts to run scripts, perhaps due to worse understanding of the current state following the history of pre-executed actions.

Can agents that reflect do better? We next evaluate if retrying after reflecting on failures can improve the performance of our baseline agents. Specifically, we rerun the ReAct-SUPER agent on all tasks where it did not submit an answer with a higher temperature. We prompt the underlying LLM to reflect (Shinn et al., 2023) on the trajectory and come up with a plan to avoid failure in the next try. This reflection is added to the prompt for the subsequent retries. We retry up to 3 times with a token budget reduced to one-third to match the token budget of ReAct-SUPER . As shown in Table 5, the additional retries with reflections have minimal impact on the score. If models lack the inherent ability to resolve some of these issues, retrial with reflections are not likely to help.

How well do agents perform on solving end-to-end tasks? We show results for the E2E in Table 4. Trends are similar, although all numbers are lower, which show the importance of scenarios to provide a better hill-climbing metric and higher-granularity for analysis.

Ablations. Our ablation study, reported in Table 6, shows that both the editing function and heuristically removing progress bars support the agent in getting more landmark outputs (64.7% with both features vs 56.7% without them). We find that without the editing command, the agent usually resorts to editing files with the `sed` command, which is designed for simple single-line edits.

The addition of the edit command allows the agent to more easily edit multiple lines, and to get a success signal of whether editing was successfull. The removal of progress bars both makes it easier for models to process long context, and perhaps more importantly, allows more interaction turns before it hits token limit.

4.3 Error Analysis

Breaking down accuracies into the different categories of the scenarios (Table 2), we find that the hardest categories are goal (23% accuracy), data (25%) and configuration (26%), whereas CPU, issues and dependencies are easier (73%, 64% and 40% respectively). These findings suggest that agents are better at solving scenarios where there is a specific error message to be solved (such as CPU support errors, incompatible dependencies, or exceptions) than more open-ended problems such as configuring data loading for a custom dataset.

Specifically, for the latter case, we find that agents commonly skip going through the repository to understand relevant code. For example, they often hallucinate arguments of scripts or functions instead of looking up how they should be called (such as adding `n_examples=10` when no such argument is defined), or the opposite: they miss a script parameter and attempt to change them in files unsuccessfully. Additionally, once they commit to a particular approach, they never reconsider their decision until failure. These failures suggest that agents should be better designed to analyze repositories and consider multiple approaches to solving any problem.

5 Conclusion

Our work introduces SUPER, a benchmark designed to evaluate LLM-based agents on executing tasks from code repositories, focusing specifically on low-profile research repositories encountered in the wild. We show empirically that our benchmark is difficult, even for the current best commercial LLMs such as GPT4, both on landmark and end-to-end task evaluations (e.g., GPT-4o solving only 38.3% of the scenarios). Our benchmark also highlights many of the core challenges in building autonomous LLM-based execution agents, such as repository reasoning and code editing, which we hope will help to make measurable progress on this important problem.

Limitations and Ethical Considerations

Dataset Size. The dataset size of our benchmark, comprising 152 scenarios, is smaller compared to some other benchmarks available for agent evaluation, which could potentially affect the statistical significance of performance evaluations. However, the use of smaller, high-quality benchmarks is not uncommon. For instance, benchmarks such as HUMANEVAL (Chen et al., 2021), CLASSEVAL (Du et al., 2023), and BAMBOOGLE (Press et al., 2023) contain 164, 100, and 125 examples respectively, and are widely used for assessing model performance. In addition, recent work has suggested that reducing large datasets to as few as 100 examples does not diminish their effectiveness (Maia Polo et al., 2024). Finally, smaller-sized datasets offer the advantage of being less expensive to operate, thus providing better accessibility for researchers with limited resources, particularly when running interactive agents in environments that generate long outputs.

Programming Languages and Domains We have only collected solutions written in Python, and our environment only supports that programming language. We focus mostly on text-based repositories. While the challenges associated with running these repositories likely overlap with other domains, increasing the diversity of the repository domains could be beneficial.

Ethical Considerations

While autonomous research execution agents could significantly enhance research advancements, there is a risk of over-reliance on these agents, which could lead to conclusions drawn based on incorrect implementations of agents, and careless actors not checking the agent's reproduction work carefully.

References

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*.

Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q Feldman, et al. 2022. Multipl-e: A scalable and extensible approach to benchmarking neural code generation. *arXiv preprint arXiv:2208.08227*.

Federico Cassano, Luisa Li, Akul Sethi, Noah Shinn, Abby Brennan-Jones, Anton Lozhkov, Carolyn Anderson, and Arjun Guha. 2023. Can it edit? evaluating the ability of large language models to follow code editing instructions. *arXiv preprint arXiv:2312.12450*.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota. Association for Computational Linguistics.

Yangruibo Ding, Zijian Wang, Wasi Ahmad, Hantian Ding, Ming Tan, Nihal Jain, Murali Krishna Ramanathan, Ramesh Nallapati, Parminder Bhatia, Dan Roth, et al. 2024. Crosscodeeval: A diverse and multilingual benchmark for cross-file code completion. *Advances in Neural Information Processing Systems*, 36.

Xueying Du, Mingwei Liu, Kaixin Wang, Hanlin Wang, Junwei Liu, Yixuan Chen, Jiayi Feng, Chaofeng Sha, Xin Peng, and Yiling Lou. 2023. Classeval: A manually-crafted benchmark for evaluating llms on class-level code generation. *arXiv preprint arXiv:2308.01861*.

Siyuan Guo, Cheng Deng, Ying Wen, Hechang Chen, Yi Chang, and Jun Wang. 2024. Ds-agent: Automated data science by empowering large language models with case-based reasoning. *ArXiv*, abs/2402.17453.

Md Mahim Anjum Haque, Wasi Uddin Ahmad, Ismini Lourentzou, and Chris Brown. 2023. Fixeval: Execution-based evaluation of program fixes for programming problems. In *2023 IEEE/ACM International Workshop on Automated Program Repair (APR)*, pages 11–18. IEEE.

Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. 2021. Measuring coding challenge competence with apps. *NeurIPS*.

Sirui Hong, Yizhang Lin, Bangbang Liu, Binhao Wu, Danyang Li, Jiaqi Chen, Jiayi Zhang, Jinlin Wang, Lingyao Zhang, Mingchen Zhuge, et al. 2024. Data interpreter: An llm agent for data science. *arXiv preprint arXiv:2402.18679*.

Qian Huang, Jian Vora, Percy Liang, and Jure Leskovec. 2024. Mlagentbench: Evaluating language agents

on machine learning experimentation. *Preprint*, arXiv:2310.03302.

Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2024. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*.

Peter Jansen, Marc-Alexandre Cot'e, Tushar Khot, Erin Bransom, Bhavana Dalvi, Bodhisattwa Prasad Majumder, Oyvind Tafjord, and Peter Clark. 2024. Discoveryworld: A virtual environment for developing and evaluating automated scientific discovery agents.

Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, Lélio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Théophile Gervet, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2024. Mixtral of experts. *Preprint*, arXiv:2401.04088.

Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*.

Y. Lai, C. Li, Y. Wang, T. Zhang, R. Zhong, L. Zettlemoyer, S. W. Yih, D. Fried, S. Wang, and T. Yu. 2023. DS-1000: A natural and reliable benchmark for data science code generation. In *International Conference on Machine Learning (ICML)*.

Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. Api-bank: A comprehensive benchmark for tool-augmented llms. In *The 2023 Conference on Empirical Methods in Natural Language Processing*.

Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. 2022. Competition-level code generation with alphacode. *Science*, 378(6624):1092–1097.

Siyi Liu, Chen Gao, and Yong Li. 2024. Large language model agent for hyper-parameter optimization. *arXiv preprint arXiv:2402.01881*.

Tianyang Liu, Canwen Xu, and Julian McAuley. 2023a. Repobench: Benchmarking repository-level code auto-completion systems. *arXiv preprint arXiv:2306.03091*.

Yuliang Liu, Xiangru Tang, Zefan Cai, Junjie Lu, Yichi Zhang, Yanjun Shao, Zexuan Deng, Helan Hu, Zengxian Yang, Kaikai An, Ruijun Huang, Shuzheng Si, Sheng Chen, Haozhe Zhao, Zhengliang Li, Liang Chen, Yiming Zong, Yan Wang, Tianyu Liu, Zhiwei Jiang, Baobao Chang, Yujia Qin, Wangchunshu Zhou, Yilun Zhao, Arman Cohan, and Mark Gerstein. 2023b. Ml-bench: Evaluating large language models for code generation in repository-level machine learning tasks. *arXiv preprint arXiv:2311.09835*.

Felipe Maia Polo, Lucas Weber, Leshem Choshen, Yuekai Sun, Gongjun Xu, and Mikhail Yurochkin. 2024. tinybenchmarks: evaluating llms with fewer examples. *arXiv preprint arXiv:2402.14992*.

OpenAI. 2023. Gpt-4 technical report. *Preprint*, arXiv:2303.08774.

OpenDevin Team. 2024. OpenDevin: An Open Platform for AI Software Developers as Generalist Agents. `https://github.com/OpenDevin/OpenDevin`. Accessed: 06/11/2024.

Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, pages 1–22.

Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah Smith, and Mike Lewis. 2023. Measuring and narrowing the compositionality gap in language models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 5687–5711, Singapore. Association for Computational Linguistics.

Yongliang Shen, Kaitao Song, Xu Tan, Wenqi Zhang, Kan Ren, Siyu Yuan, Weiming Lu, Dongsheng Li, and Yueting Zhuang. 2023. Taskbench: Benchmarking large language models for task automation. *arXiv preprint arXiv:2311.18760*.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*.

Wilson L Taylor. 1953. "cloze procedure": A new tool for measuring readability. *Journalism quarterly*, 30(4):415–433.

Runchu Tian, Yining Ye, Yujia Qin, Xin Cong, Yankai Lin, Zhiyuan Liu, and Maosong Sun. 2024. Debugbench: Evaluating debugging capability of large language models. *arXiv preprint arXiv:2401.04621*.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023a. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*.

Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. 2024. Executable code actions elicit better llm agents. *arXiv preprint arXiv:2402.01030*.

Xingyao Wang, Zihan Wang, Jiateng Liu, Yangyi Chen, Lifan Yuan, Hao Peng, and Heng Ji. 2023b. Mint: Evaluating llms in multi-turn interaction with tools and language feedback. *arXiv preprint arXiv:2309.10691*.

Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. 2024. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *arXiv preprint arXiv:2404.07972*.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024a. Swe-agent: Agent-computer interfaces enable automated software engineering. *Preprint*, arXiv:2405.15793.

Zhiyu Yang, Zihan Zhou, Shuo Wang, Xin Cong, Xu Han, Yukun Yan, Zhenghao Liu, Zhixing Tan, Pengyuan Liu, Dong Yu, et al. 2024b. Matplotagent: Method and evaluation for llm-based agentic scientific data visualization. *arXiv preprint arXiv:2402.11453*.

Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. 2022. Webshop: Towards scalable real-world web interaction with grounded language agents. *Advances in Neural Information Processing Systems*, 35:20744–20757.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*.

Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. 2023. Repocoder: Repository-level code completion through iterative retrieval and generation. *arXiv preprint arXiv:2303.12570*.

Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, et al. 2023. Webarena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*.

A Repository details

Table 7 shows information about the 42 source repositories used to create the problems in the benchmark, including their name (**task_id**), original Github link (**github repo**) and the number of stars on GitHub (**stars**).

Adding to the information in Table 1, we show below the average and median star ratings for other comparable benchmarks (all star ratings are collected as of `June 14, 2024`). This was computed automatically from the GitHub API based on the repositories listed in Jimenez et al. (2024), Lai et al. (2023) and Liu et al. (2023b) (we group together both the train and test repositories mentioned in this table).

dataset	**# repos**	**stars** *(mean (median))*
swe-bench	12	27,449 (12,308)
DS1000	8	54,336 (34,589)
MLBench	18	12,620 (8,991)
SUPER	42	219 (11)

B Instructions to Experts

Workers were hired through Upwork (`upwork.com`), were paid $30-$40/hour, and were limited to four hours per task, although in some cases they were approved for up to an additional 2 hours if the task wasn't completed in time. In a few cases, the experts were unable to run the experiments due to CoLab or dependencies issues; these tasks were discarded. In total, task collection cost $6,580 for 46 solutions, of which we keep the final set of 42 tasks.

We provide the instructions given to the Upwork experts in Figs. 4 to 6.

C Prompts and Interaction History

We provide the prompt used for ReAct-`SUPER` in Appendix C. We use the JSON decoding feature of OpenAI and `together.ai` to ensure correct format of outputs.

The demonstrations used in this prompt were not taken from `SUPER`, and were designed specifically to show the basic usage of the available actions.

task_id	github repo	stars
colbert	https://github.com/stanford-futuredata/ColBERT	2607
textbox	https://github.com/RUCAIBox/TextBox	1066
amrbart	https://github.com/goodbai-nlp/AMRBART	89
g-transformer	https://github.com/baoguangsheng/g-transformer	44
pie-perf	https://github.com/madaan/pie-perf	79
safetybench	https://github.com/thu-coai/SafetyBench	116
discodisco	https://github.com/gucorpling/DisCoDisCo	5
acqsurvey	https://github.com/rahmanidashti/acqsurvey	9
curriculum_learning	https://github.com/adymaharana/curriculum_learning	9
spa	https://github.com/OceannTwT/SPA	4
mezo	https://github.com/princeton-nlp/MeZO	990
mode-connectivity-plm	https://github.com/thunlp/mode-connectivity-plm	8
mbib	https://github.com/Media-Bias-Group/MBIB	20
quantifying-stereotypes-in-language	https://github.com/nlply/quantifying-stereotypes-in-language	1
rah-kbqa	https://github.com/yanmenxue/rah-kbqa	6
dir-gnn	https://github.com/wuyxin/dir-gnn	114
unsupervised hierarchical symbolic regression	https://github.com/SiyuLou/UnsupervisedHierarchicalSymbolicRegression	0
conv_graph	https://github.com/huawei-noah/noah-research/tree/master/conv_graph	0
mera	https://github.com/ai-forever/MERA	48
pira	https://github.com/C4AI/Pira	5
pet	https://github.com/timoschick/pet	1611
transnormerllm	https://github.com/opennlplab/transnormerllm	218
bert-lnl	https://github.com/uds-lsv/BERT-LNL	9
blockskim	https://github.com/chandlerguan/blockskim	6
data_label_alignment	https://github.com/gyauney/data-label-alignment	3
hype	https://github.com/yuanhy1997/HyPe	13
paraphrase-nli	https://github.com/matejklemen/paraphrase-nli	3
powerfulpromptft	https://github.com/zhengxiangshi/powerfulpromptft	71
robust_prompt_classifier	https://github.com/adianliusie/robust-prompt-classifier	5
align-to-distill	https://github.com/ncsoft/Align-to-Distill	4
inbedder	https://github.com/zhang-yu-wei/InBedder	15
transpolymer	https://github.com/ChangwenXu98/TransPolymer	47
memorizing-transformers-pytorch	https://github.com/lucidrains/memorizing-transformers-pytorch	617
multi3woz	https://github.com/cambridgeltl/multi3woz	14
galore	https://github.com/jiaweizzhao/galore	1202
amos	https://github.com/microsoft/amos	24
glee	https://github.com/genezc/Glee	9
parallel-context-windows	https://github.com/AI21Labs/Parallel-Context-Windows	91
logme-nlp	https://github.com/mainlp/logme-nlp	5
mixup-amp	https://github.com/pai-smallisallyourneed/mixup-amp	4
upet	https://github.com/wjn1996/UPET	2
dpt	https://github.com/xyaoooo/dpt	6

Table 7: Details of the 42 repositories used in SUPER along with GitHub link and star information.

Project goal

The goal of this project is for you to create Jupyter Notebooks that solve ML/NLP-related problems, which we refer to as "tasks." Each assigned task will include a description of the problem to be solved, a GitHub repository to use when solving that problem, and any additional instructions that are needed. An example task and solution (notebook) is included at the end of this document.

Please read this document in full before beginning your first task.

Materials

You will be provided with:

- **A Google Drive folder** - this is where you should create your Jupyter Notebooks as Google Colab files. A screenshot of how to create a Google Colab notebook is included at the end of this document.

- **A Google Sheet** - this is where we will enter the descriptions for each of your assigned tasks. There is space in this spreadsheet where you are asked to enter key information for each task:
  - pip dependencies
  - current repository commit
  - your resulting answer
  - a link to your notebook
  - Task reproducibility comments (which should include any details that are necessary to reproduce your results)
  - **Notebook estimated e2e runtime**: the amount of time it takes to run the entire notebook end to end, in minutes. We generally want this to be less than 5-7 minutes. If it is more than that, please see if some hyperparameters can be changed to reduce time (e.g. less epochs, data etc.) and consult with us if necessary.
  - the amount of time you spent on each task. Ensure that you update this spreadsheet as you work through each task.

Time considerations

Please **closely track your time spent on each task,** and indicate the time spent in your task description spreadsheet. Some tasks will have to be discarded due to taking an excessive amount of time, so please keep the following points in mind as you work through your assigned tasks:

- **Task setup:** It may be difficult to get research repositories to work properly in a Google Colab setting due to dependencies that changed or other environment issues. For this reason, we will cap the amount of time spent on dependencies setup (solving installation and versions conflict) and ask you to skip tasks where the installation time is over 1 hour. **Do not exceed 60 minutes of task setup unless we have approved you to do so.**

- **Reaching a solution:** If you have spent a total of 4 hours on a task and have not reached a solution, please pause your work on that task and let us know. We will review the work you have completed so far to determine whether or not to have you continue working towards a solution on that task. **Do not exceed 4 hours of work on a single task unless we have approved you to do so.**

Figure 4: Guidelines provided to experts.

Please note that you will be paid for your total time spent, not for the number of tasks completed, so **ensure you track your time spent.**

Instructions

- **Your work should be submitted as a Jupyter Notebook in Google Colab.** Please create your notebook in the Google Drive folder that was shared with you, making sure that we can re-run everything to get the same result (e.g. don't delete or edit cells that need to be run for things to work.) Add the link to each task's notebook to your task description spreadsheet.

- **Different people completing the same task should get identical results.** Wherever possible, use default configurations. If you've made any decision where you think others might choose differently, please state so in the task comments. For example, if there is no default batch-size value, and you chose it to be 8, add this to the task comments in your task description spreadsheet.

- **Add pip dependencies and current repository commit to the spreadsheet.** This helps us to ensure we can reproduce your notebook in the future. Specifically:
  - To fill the dependencies column, paste the output of `!pip list` (make sure you run this after you finish the task)
  - To fill the repository commit column, paste the output of `!git rev-parse HEAD`

- **Google Colab has some environmental limitations.** For example, Conda cannot be easily installed, and RAM may be limited. Use simple means to overcome such limitations, specifically:
  - Instead of using Conda, read the Conda environment yaml file, and install required packages through pip. You can also download the miniconda installation file and install it. However, please **do not** use solutions that are specific to Colab such as condacolab.
  - Insufficient RAM/resources: this can often be overcome by reducing the size of data you work on, or by reducing the batch size. If you do that, include your change in the task comments in your task description spreadsheet
  - CPU runs: please run all notebooks on CPU, not GPU.
  - Long runs: if it takes more than a few minutes to run a cell, follow the same instructions mentioned for insufficient RAM/resources.

  If limitations cannot be overcome without changing the task, please consult with us.

- **Research repositories often have limitations.** Some repositories might have missing code, or unexpected issues. Do your best to overcome these, and consult with us if you do not think that would be possible. You may need to modify the code as we show next.

- **Change repository code within Colab cells.** One easy way to do this is by cloning the repository on your own machine, changing the code as you need, then creating a patch for your changes (e.g. with `git diff > changes.patch` or through your IDE). Once you have the patch, you can apply it in Colab using `%%writefile` and `git apply my_patch.patch`, as seen here:

```
[20] %%writefile my_patch.patch
     Index: colbert/parameters.py
     IDEA additional info:
     Subsystem: com.intellij.openapi.diff.impl.patch.CharsetEP
     <+>UTF-8
     ===================================================================
     diff --git a/colbert/parameters.py b/colbert/parameters.py
     --- a/colbert/parameters.py (revision 02f3d077588af8aab2b384884fc28fb89c1e
     +++ b/colbert/parameters.py (date 1710267992256)
     @@ -1,6 +1,6 @@
      import torch

     -DEVICE = torch.device("cuda")
     +DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

      SAVED_CHECKPOINTS = [32*1000, 100*1000, 150*1000, 200*1000, 300*1000, 400
      SAVED_CHECKPOINTS += [10*1000, 20*1000, 30*1000, 40*1000, 50*1000, 60*100

     Overwriting my_patch.patch
```

Figure 5: Guidelines provided to experts (continued).

```
[21] !git apply my_patch.patch
```

As an alternative, you can also use bash commands such as `sed` to edit files, if you prefer.

- **Changes to the repository should be minimal.** Where possible, prefer to write code in notebook cells rather than changing the repository code. For example, if the repository code expects a csv file but you have a json file, convert the json file to a csv file in a notebook cell, rather than editing the code to accept json.

- **Follow the readme instructions wherever possible**.

- **You can use other repositories and packages if needed**, but only use well-known repositories. For example, you could use:
  - HuggingFace datasets, numpy, torch, sklearn, and other popular ML-related packages.
  - gdown to download dataset files from the google drive links we provide.
  - If you are unsure whether a repo/package is appropriate to use, please consult with us.

- **We should be able to run your notebook effortlessly with the "Run All" command**.
  - Please ensure that there will be no user prompts when running the commands, e.g.

```
files2rouge uses scripts and tools that will not be stored with the python package
where do you want to save it? [default: /root/.files2rouge/]
```

  You can bypass such prompts using the pipe command (e.g. `echo "answer" | python prompt.py`

Figure 6: Guidelines provided to experts (continued).

Agent prompt

```
Interact in a Jupyter notebook to solve the request.
Output a thought and an action.
Thought can reason about the current situation, and Action allows you to interact with the Jupyter notebook. There are three types of actions:

(1) execute, which executes a notebook cell with the command under the "content" field. This could be a bash command (beginning with !), a python command, or magic command (beginning with %).
(2) edit, which allows you to replace existing lines in a file with new lines. To replace lines, you will need to provide the lines as they appear before the replacement, and the lines to replace them with.
In the "content" field, format should be "‘{filename}
<BEFORE_EDIT>
lines before edit
lines before edit
<AFTER_EDIT>
lines after edit
lines after edit"‘
Follow the json format, always output exactly "thought" and "action".
I will provide the cell output (observation) for you.

Tips:
- When a github repository is given, you should start by cloning it. You can read the readme file and browse the repo for installation and usage instructions.
- Avoid using conda, instead use pip for package management.
- Each execution has a timeout of 5 minutes, after each you will be able to decide whether to interrupt the kernel or continue to wait.
You should interrupt if (1) you are prompted with an interactive message, (such as "Do you want to continue? [Y/n]"). You can then bypass it e.g. by executing ‘echo "y" | command‘ or (2) running is taking more than expected since the wrong thing is being executed, e.g. a different model or wrong number of epochs.
You should continue to wait if the execution that is running is the correct one, and is working as expected.

Here are some examples.
Git repository: EleutherAI/lm-evaluation-harness
Query: Evaluate EleutherAI/gpt-j-6B on hellaswag. What's the score?
{"thought": "First, I need to clone the GitHub repository 'EleutherAI/lm-evaluation-harness'.", "action": {"type": "execute", "content": "!git clone https://github.com/EleutherAI/lm-evaluation-harness.git"}}
{"observation": "Cloning into 'lm-evaluation-harness'..."}
{"thought": "The repository has been cloned. Next, I should check the README file for installation and usage instructions to proceed with evaluating EleutherAI/gpt-j-6B on hellaswag.", "action": {"type": "execute", "content": "!cat lm-evaluation-harness/README.md"}}
{"observation": " Language Model Evaluation Harness ... truncated ..."}
{"thought": "The README provides detailed instructions for installing the ‘lm-eval‘ package and evaluating models like GPT-J-6B on tasks like hellaswag. I need to install the package and its dependencies first.", "action": {"type": "execute", "content": "!pip install -e lm-evaluation-harness/."}}
{"observation": "Installing build dependencies ... truncated ..."}
{"thought": "The ‘lm-eval‘ package and its dependencies have been installed successfully. Now, I can proceed with evaluating the EleutherAI/gpt-j-6B model on the hellaswag task as per the instructions in the README.", "action": {"type": "execute", "content": "!lm_eval --model hf --model_args pretrained=EleutherAI/gpt-j-6B --tasks hellaswag --device cuda:0 --batch_size 8"}}
{"observation": "{"hellaswag": {"acc": 0.24603658536585363,"acc_norm": 0.34436059706446426,}, ... truncated ..."}}
Query: use pandas to read the file 'data.csv' and display the first 2 rows.
{"thought": "First, I need to read the file 'data.csv' using the pandas library.", "action": {"type": "execute", "content": "import pandas as pd.read_csv('data.csv').head()"}}
{"observation": " name0 Alice1 Bob2 Charlie"}
{"thought": "The file 'data.csv' has been read successfully and the first 2 rows have been displayed.", "action": {"type": "submit", "content": ["Alice", "Bob"]}}

Query: Change the model name in the 'config.json' file from 'roberta-base' to 'roberta-large'.
{"thought": "First, I need to read the 'config.json' file to find the location where I can replace the model name.", "action": {"type": "execute", "content": "!cat config.json"}}
{"observation": "{"model": "roberta-base","batch_size": 8,"epochs": 10}"}
{"thought": "The file has been read successfully. Now, I can replace the model name in the 'config.json' file.", "action": {"type": "edit", "content": "config.json
<BEFORE_EDIT>
{
"model": "roberta-base",
"batch_size": 8,
<AFTER_EDIT>
{
"model": "roberta-large",
"batch_size": 8,
"}}"}}
{"observation": "File config.json has been edited succesfully."}
{"thought": "The model name has been replaced in the 'config.json' file.", "action": {"type": "submit", "content": null}}
```

History of Interactions Following the above prompt, at each step we provide the history of all past interactions by combining the JSON representations of the (thought, action, observation) tuple into a single string, given as a single 'message' to the LLM API:

```
\{\{"thought": \{thought\}, "action": \{action\}\}\}\n'
\{\{"observation": \{observation\}\}\}\n'
```

Pre-executed steps are also be part of this inter-

action history, with a fixed thought:

```
[pre-executed by the user]
```

Since trajectories can often get long (§4.1), we use the following truncation strategy for all the ReAct agents: for the last step, we provide the 50k last characters, which is usually enough for the entire observation. For earlier steps, we shorten the observations to show the last Y=500 characters.

D Edit command

The edit command is an action that agents can select at every iteration, in addition to execute and command. The format of the input to this command is as follows:

```
{filename}
<BEFORE_EDIT>
(lines before edit)
<AFTER_EDIT>
(lines after edit)
```

Where `(lines before edit)` are the exact lines to be replaced, and `(lines after edit)` are the lines to replace them with.

To provide the exact file contents that should be replaced, agents typically need to view the existing contents of the file, for example by using the `cat` command, and then copy it verbatim as the lines to be replaced.[3] Our environment then looks for the contents to be replaced in the file and replaces it with the new contents.

The edit command requires the provided replaced lines to be (1) precisely copied, including correct whitespaces and indentations and (2) unique in the contents file, so that the edit command is not ambiguous. To help the agent with these requirements, we configure the edit command to provide specific feedback to the agent in case one of these conditions does not apply.

Specifically, if the the lines to be replaced were not found as-is, but these lines do appear in the edited file *without* surrounding whitespaces or tabs, then the environment provides the following feedback:

```
Did you mean to replace the following
lines (notice leading/trailing
whitespaces difference)?
```

followed by an exact copy of these lines, including the spaces.

If more than one instances of these lines were found, the following feedback is provided:

```
Found multiple ([k]) occurrences of the
<BEFORE_EDIT> lines. Add 1-3 lines
before or after these lines to replace
to disambiguate.
Here are the first two occurrences with
additional context, did you mean one of
these?
Occurrence 1: ...
```

with the first two occurences of these lines.

[3] Solutions that accept line numbers instead of exact content performed worse in our early experiments.